

17. Profiles và Environment trong Spring Boot

17.1. Giới thiệu

Trong quá trình phát triển và triển khai ứng dụng, chúng ta thường cần các cấu hình khác nhau cho các môi trường khác nhau (ví dụ: môi trường phát triển, kiểm thử, sản xuất). Ví dụ, trong môi trường phát triển, bạn có thể muốn sử dụng cơ sở dữ liệu nhúng H2 và logging chi tiết, trong khi ở môi trường sản xuất, bạn cần kết nối với cơ sở dữ liệu MySQL thực và logging ở mức độ ít chi tiết hơn.

Spring Boot cung cấp tính năng **Profiles** để giải quyết vấn đề này một cách hiệu quả. Profiles cho phép bạn định nghĩa các cấu hình khác nhau và kích hoạt chúng tùy thuộc vào môi trường mà ứng dụng đang chạy.

17.2. Khái niệm Profiles

Profile là một định danh logic mà bạn có thể gán cho các bean, các thuộc tính cấu hình, hoặc thậm chí toàn bộ các lớp cấu hình. Khi một profile được kích hoạt, chỉ những bean hoặc cấu hình được liên kết với profile đó mới được tải và áp dụng.

17.2.1. Định nghĩa Profile-specific Properties

Cách phổ biến nhất để sử dụng profiles là tạo các file cấu hình riêng cho từng profile. Spring Boot sẽ tự động nhận diện các file có tên theo định dạng `application-{profile}.properties` hoặc `application-{profile}.yaml`.

Ví dụ:

- `src/main/resources/application.yaml` (cấu hình chung)
- `src/main/resources/application-dev.yaml` (cấu hình cho môi trường phát triển)
- `src/main/resources/application-prod.yaml` (cấu hình cho môi trường sản xuất)

Khi một profile được kích hoạt, các thuộc tính trong file profile-specific sẽ ghi đè lên các thuộc tính có cùng tên trong file `application.yaml` chung.

17.2.2. Định nghĩa Profile-specific Beans và Configuration

Bạn có thể sử dụng annotation `@Profile` để đánh dấu các bean hoặc các lớp `@Configuration` chỉ được tạo khi một profile cụ thể được kích hoạt.

Ví dụ:

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

@Configuration
@Profile("dev") // Chỉ kích hoạt khi profile 'dev' được bật
public class DevConfig {

    @Bean
    public String devMessage() {
        return "This is a development message.";
    }
}

@Configuration
@Profile("prod") // Chỉ kích hoạt khi profile 'prod' được bật
public class ProdConfig {

    @Bean
    public String prodMessage() {
        return "This is a production message.";
    }
}

@Configuration
@Profile({"dev", "test"}) // Kích hoạt khi profile 'dev' HOẶC 'test' được bật
public class DevAndTestConfig {

    @Bean
    public String devTestMessage() {
        return "This message is for dev or test environments.";
    }
}

@Configuration
@Profile("!prod") // Kích hoạt khi profile 'prod' KHÔNG được bật
public class NonProdConfig {
```

```
@Bean
public String nonProdMessage() {
    return "This message is for non-production environments.";
}
}
```

17.3. Kích hoạt Profiles

Có nhiều cách để kích hoạt một hoặc nhiều profile trong Spring Boot, với thứ tự ưu tiên khác nhau:

17.3.1. Qua `application.properties` hoặc `application.yml`

Bạn có thể đặt thuộc tính `spring.profiles.active` trong file cấu hình chính:

YAML

```
spring:
  profiles:
    active: dev
```

Hoặc nhiều profile:

YAML

```
spring:
  profiles:
    active: dev,h2
```

17.3.2. Qua Command Line Arguments

Đây là cách phổ biến nhất để kích hoạt profile khi chạy ứng dụng:

Bash

```
java -jar myapp.jar --spring.profiles.active=prod
```

Để kích hoạt nhiều profile:

Bash

```
java -jar myapp.jar --spring.profiles.active=prod,cloud
```

17.3.3. Qua Environment Variables

Bạn có thể đặt biến môi trường `SPRING_PROFILES_ACTIVE` :

Bash

```
export SPRING_PROFILES_ACTIVE=test  
java -jar myapp.jar
```

17.3.4. Qua Web.xml (đối với ứng dụng WAR)

Đối với các ứng dụng được đóng gói dưới dạng WAR và triển khai trên một máy chủ ứng dụng truyền thống, bạn có thể cấu hình trong `web.xml` :

XML

```
<context-param>  
  <param-name>spring.profiles.active</param-name>  
  <param-value>prod</param-value>  
</context-param>
```

17.3.5. Qua System Properties

Bash

```
java -Dspring.profiles.active=prod -jar myapp.jar
```

Thứ tự ưu tiên khi kích hoạt Profiles:

Các phương pháp kích hoạt profile có thứ tự ưu tiên. Phương pháp có độ ưu tiên cao hơn sẽ ghi đè phương pháp có độ ưu tiên thấp hơn:

1. `spring.profiles.active` trong `application.properties` (hoặc `application.yml`) được chỉ định bởi `spring.config.location` .

2. `SPRING_PROFILES_ACTIVE` (biến môi trường).
3. `spring.profiles.active` (System Property).
4. `--spring.profiles.active` (Command Line Argument).

17.4. Environment trong Spring Boot

Đối tượng `Environment` trong Spring Framework là một giao diện trung tâm để truy cập các thuộc tính cấu hình và profiles đang hoạt động. Nó cung cấp một cách thống nhất để truy cập các nguồn thuộc tính khác nhau (file properties, biến môi trường, system properties, v.v.).

17.4.1. Inject Environment

Bạn có thể inject đối tượng `Environment` vào bất kỳ Spring bean nào:

Java

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.env.Environment;
import org.springframework.stereotype.Component;

@Component
public class AppConfigReader {

    @Autowired
    private Environment env;

    public void printAppInfo() {
        System.out.println("App Name: " +
            env.getProperty("spring.application.name"));
        System.out.println("Server Port: " + env.getProperty("server.port"));
        System.out.println("Active Profiles: " + String.join(", ",
            env.getActiveProfiles()));

        if
            (env.acceptsProfiles(org.springframework.core.env.Profiles.of("dev"))) {
                System.out.println("Running in Development mode.");
            }
    }
}
```

17.4.2. PropertySources

`Environment` chứa một tập hợp các `PropertySource`, mỗi `PropertySource` đại diện cho một nguồn thuộc tính (ví dụ: `application.yml`, biến môi trường, `system properties`). Các `PropertySource` được sắp xếp theo thứ tự ưu tiên, với các nguồn có độ ưu tiên cao hơn sẽ được tìm kiếm trước.

Bạn có thể thêm các `PropertySource` tùy chỉnh nếu cần.

17.5. Bản chất của Profiles và Environment

Bản chất của Profiles và Environment trong Spring Boot là cung cấp một cơ chế mạnh mẽ và linh hoạt để quản lý cấu hình ứng dụng theo từng môi trường. Thay vì phải thay đổi mã nguồn hoặc đóng gói lại ứng dụng cho mỗi môi trường, Profiles cho phép bạn định nghĩa các cấu hình khác nhau và kích hoạt chúng một cách động. Điều này thúc đẩy nguyên tắc "Externalized Configuration" (cấu hình bên ngoài), giúp ứng dụng trở nên linh hoạt, dễ triển khai và bảo trì hơn trong các môi trường khác nhau, từ phát triển đến sản xuất. Sự kết hợp giữa Profiles và đối tượng `Environment` cung cấp một giải pháp toàn diện để quản lý các biến thể cấu hình, đảm bảo rằng ứng dụng luôn hoạt động đúng cách trong mọi ngữ cảnh.